Procedural Dungeon Roguelike — Technical Documentation

Oct 23, 2026 · @Syed Jafri

Overview

A turn-based, grid-based roguelike where every run is fully reproducible from one seed: the same seed and the same inputs always produce the same dungeon, the same rolls and the same outcome.

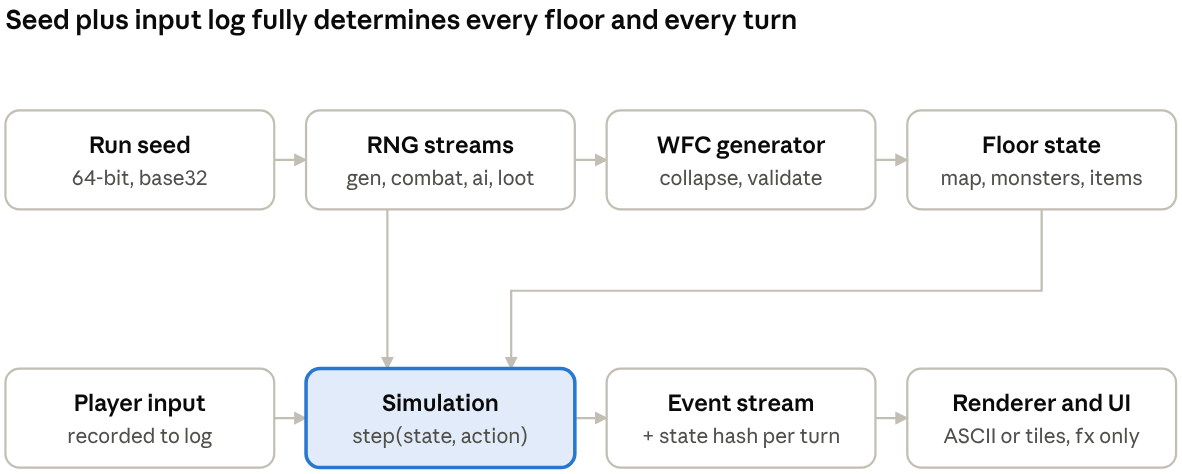
Design pillars

- **Deterministic** — seed + input log = exact replay. Enables shareable seeds, daily challenges, bug reports that reproduce, and replay-based tests.
- **Generated, but authored** — wave-function collapse (WFC) assembles levels from hand-made tiles and adjacency rules, so layouts feel designed rather than noisy.
- **Data-driven** — abilities, status effects, monsters and items are defined in data files composed from small effect primitives; adding content needs no engine code.

Scope (v1) — single-player, 4-direction + diagonal grid movement, 10 floors plus a boss floor, permadeath, ASCII and tile renderers. Out of scope: multiplayer, real-time modes, and modding of engine code (data mods are supported).

Architecture

The engine splits into a deterministic core (seed, RNG, generation, simulation) and a non-deterministic shell (input, rendering, audio) that never feeds back into game state except through recorded player actions.



system architecture · seed to screen

The compiled content pack (abilities, monsters, tilesets) is loaded by both the generator and the simulation. Suggested stack: Rust core library (`dungeon-core`, no I/O, `#![forbid(unsafe_code)]`), a thin client in Bevy or a terminal front end, and WebAssembly builds sharing the same core for browser play and server-side replay verification.

Determinism

All randomness comes from named, independent RNG streams derived from the run seed, and the simulation uses only integer math, so results are identical across platforms, builds and replays.

Seed — a 64-bit integer; players see it as a 13-character Crockford base32 string (e.g. `K7Q2-M9XD-4TRA`). Daily seeds = hash("daily", UTC date).

RNG — PCG32 (or xoshiro256**) implemented in-house, never the language or engine RNG. Each stream is seeded by hashing (run seed, stream name, index) with SplitMix64:

Stream	Key	Used for
`gen.floor`	floor number	WFC collapse order and tile choice
`gen.populate`	floor number	Monster, item, trap placement
`combat`	—	Hit, damage, crit rolls
`ai`	—	Monster decision tie-breaks
`loot`	—	Drop tables

Stream	Key	Used for
`fx`	—	Cosmetic only (particles, flavour text); never read by simulation

Separate streams mean that, for example, adding a cosmetic effect or a new monster on floor 5 does not change floors 1–4 or combat rolls elsewhere.

Rules for simulation code (enforced by lint and review)

1. No floating point in game state; use integers or fixed-point (16.16).
2. No wall-clock time, frame time, thread scheduling or hash-map iteration order in logic; use ordered maps and sorted entity IDs.
3. Entities update in a fixed order (by initiative, then entity ID).
4. Every random call names its stream; the presentation layer may only use `fx`.
5. Save files store the seed, stream states and the input log, not derived state alone.

State hash — after each turn, a 64-bit hash of game state (FNV-1a over a canonical serialization) is recorded. Replays compare hashes turn by turn and report the first divergent turn.

Wave-function-collapse level generation

Each floor is generated in three passes — constraints, WFC collapse, then validation and population — and is rejected and retried with the next sub-seed if it fails connectivity checks.

Model — simple tiled WFC. A tileset defines modules (e.g. 3×3 cells: room interior, wall, corridor, door, corner), each with a socket label per edge and a weight. Two modules may be adjacent when their facing sockets match. Rotations and mirrors are generated automatically unless a module sets `symmetry: none`.

Pass 1 — constraints. Before collapse, cells are pre-set or restricted:

- Border cells limited to wall modules.
- Stairs-down and stairs-up placed at seeded positions at least `gen.min_stair_distance` apart (default 60% of the map diagonal).
- Optional set pieces (vaults, boss arena) stamped as fixed regions.

Pass 2 — collapse.

1. Each cell holds a bitset of still-possible modules.
2. Pick the uncollapsed cell with lowest Shannon entropy; ties broken by the `gen.floor` stream, then by cell index (for determinism).
3. Choose one module weighted by `weight`, using `gen.floor`.
4. Propagate: remove incompatible modules from neighbours with an AC-3–style queue until stable.
5. On contradiction (a cell with no options), backtrack up to `gen.max_backtrack` (default 64) decisions; if exhausted, restart with sub-seed + 1.

Pass 3 — validation and population.

- Flood fill from stairs-up: stairs-down must be reachable; unreachable pockets are filled with wall or, if large, connected with a carved corridor.
- Checks: walkable area 35–60% of the map, at least `gen.min_rooms` rooms, no dead-end corridors longer than 12 tiles.
- Populate from `gen.populate`: monsters by floor budget (sum of monster `threat` ≤ budget), items from loot tables, traps away from the stair paths.

Performance target — a 64 × 48 floor generates in < 50 ms median, < 250 ms p99 including retries; retries above 5 are logged as a tileset quality issue.

Turn system and simulation

Time advances in discrete ticks on an energy scheduler; the simulation is a pure function `step(state, action) → (state', events)`, and the renderer only consumes events.

Energy scheduler — each actor has `speed` (default 100). Every tick, actors gain `speed` energy; any actor at ≥ 1000 energy acts, then pays the action's cost (move 1000, attack 1000, quick actions 500, heavy 1500). Ties resolve by higher energy, then lower entity ID. The game pauses for input only when the player is the next actor.

Action pipeline

1. **Validate** — is the action legal (target in range, resources available, not stunned)?
2. **Pre-hooks** — status effects and passives may modify or cancel it (e.g. *Confused* redirects movement).
3. **Resolve** — run the action's effects (see ability system); all rolls on the `combat` stream.
4. **Post-hooks** — reactions such as counterattacks or on-hit procs, depth-limited to 8 to prevent loops.

5. **Emit events** — `Moved`, `Damaged`, `Died`, `StatusApplied`, etc., consumed by the log, renderer and audio.
6. **End of turn** — tick status durations, regen, then record the state hash.

Field of view — symmetric recursive shadowcasting, radius 8 by default; monsters use the same algorithm so "I can see you" implies "you can see me".

AI — monsters choose actions with a utility scorer over their available abilities (data-defined considerations such as distance, HP %, ally count). Pathfinding uses A* with a Dijkstra map cache per floor for flee and approach behaviours. Ties use the `ai` stream.

Saving — saves store the seed, RNG stream states, input log and a full state snapshot every 100 turns for fast loading; loading verifies the snapshot against a replay of the log tail.

Data-driven ability system

An ability is data: a cost, a targeting shape, and an ordered list of effects built from a small set of engine primitives. Player skills, monster attacks, item uses and traps all share this format.

Ability definition

```
id: fire_lance
name: Fire Lance
tags: [spell, fire, ranged]
cost: {mana: 12, energy: 1000}
cooldown: 4
targeting:
  shape: line          # self | single | line | cone | burst | ring
  range: 6
  requires_los: true
  pierce: 2
effects:
  - damage: {amount: "2d6 + INT/2", type: fire}
  - apply_status: {status: burning, duration: 3, chance: 40}
  - when: {target_has_tag: frozen}
    do:
      - remove_status: {status: frozen}
      - damage: {amount: "1d6", type: fire}
ai:
  considerations: [target_in_line, enemies_hit_count]
```

Effect primitives

Primitive	Does
`damage`	Dice expression × resistances → HP loss; emits `Damaged`
`heal`	Restore HP, capped at max
`apply_status` / `remove_status`	Add or remove a status by id
`move`	Push, pull, teleport, swap, dash along path
`spawn`	Create entity or terrain (fire, ice, summon)
`modify_stat`	Temporary or permanent stat change
`trigger_ability`	Chain into another ability (depth-limited)
`when` / `do`	Condition gate over target/caster tags, stats, rolls
`repeat`	Run nested effects N times

Expressions — damage and chance values use a tiny integer expression language: dice (`2d6`), stat references (`INT` , `LVL`), `+ - × /` with integer division, `min` / `max` . Parsed once at load time to bytecode.

Statuses — also data: `duration` , `stacking` (refresh | add | max), tick effects, and hooks (`on_apply` , `on_turn_start` , `on_hit` , `on_damaged` , `on_expire`) that run effect lists. Hooks are how passives, auras and reactions work without engine code.

Validation — on load, every file is checked against a JSON Schema plus semantic rules: unknown ids, unreachable `when` branches, cycles in `trigger_ability` , and expressions that reference missing stats. Content errors fail the build with file and line.

Content formats, tooling and CLI

All game content lives in versioned YAML under `content/` , is validated and compiled into one binary pack at build time, and can be overridden by data mods loaded in a fixed order.

Layout

```
content/
  abilities/*.yaml
  statuses/*.yaml
  monsters/*.yaml        # stats, abilities, AI considerations, threat
  items/*.yaml           # equipment, consumables (use = ability id)
  loot_tables/*.yaml
  tilesets/<name>/modules.yaml + sprites/
  floors/*.yaml          # per-floor tileset, size, budget, set pieces
mods/<mod_id>/...        # same layout; later mods override by id
```

Floor definition example

```
floor: 3
tileset: crypt
size: [64, 48]
threat_budget: 42
min_rooms: 6
set_pieces: [{id: ossuary_vault, chance: 30}]
monster_pool: [skeleton, ghoul, bone_archer]
```

CLI

```
dungeon validate content/                # schema + semantic checks
dungeon build    content/ -o content.pack
dungeon gen      --seed K7Q2-M9XD-4TRA --floor 3 --render ascii
dungeon gen      --seeds 1:10000 --floor 3 --stats   # retries, area, timing
dungeon replay   run.replay --verify              # compare state hashes
dungeon sim      --seed K7Q2-M9XD-4TRA --bot greedy --turns 5000
```

Debug tools (in-game, dev builds) — WFC step-through viewer (entropy heatmap, propagation highlights), seed/floor jump, ability inspector showing resolved expressions, and a turn-by-turn state hash overlay.

Testing, limitations and roadmap

Determinism makes the test suite mostly replays: a recorded seed and input log must reproduce the same state hash on every platform and every commit.

Testing

- Golden replays: a corpus of recorded runs; CI replays each on Windows, macOS, Linux and WebAssembly and requires identical hash sequences.

- Generation sweeps: 10 000 seeds per floor definition; fail on unreachable stairs, retry rate > 1%, or p99 time over budget.
- Bot soak tests: scripted bots play thousands of turns per seed to catch crashes, infinite reaction loops and softlocks.
- Content tests: every ability is executed against a test dummy and its emitted events compared to a snapshot.
- Unit tests for RNG streams (known outputs per seed), WFC propagation, expression parser and energy scheduler.

Known limitations

- Changing content or engine code changes outcomes for existing seeds; replays and shared seeds are tied to a content-pack version, stored in every save and replay.
- Simple tiled WFC can produce repetitive local patterns with small tilesets; set pieces and weighting mitigate this.
- Large maps (> 128 × 128) make backtracking expensive; v1 caps floor size.
- Integer-only math limits some formulas (e.g. smooth falloff) to lookup tables.

Roadmap

- [ ] Overlapping-model WFC trained from hand-drawn sample maps
- [ ] Multi-floor constraints (persistent shafts, linked vaults)
- [ ] Visual ability editor generating YAML
- [ ] Daily-seed leaderboard with server-side replay verification
- [ ] Mod workshop with signed content packs